Knowledge Base Dua Lapis Berversi Waktu untuk Perhitungan PPh Pasal 21: Rancangan Sistem, Aplikasi Web, dan Metodologi Validasi tanpa *Domain Expert*

Draf—sampai metodologi penelitian; hasil eksperimen belum disertakan

[Nama Penulis]
Program Magister Informatika
[Nama Perguruan Tinggi]
[Kota], Indonesia
[email]

***Abstrak*—Perhitungan PPh Pasal 21 pegawai tetap berubah hampir setiap tahun. Sejak 2024 metode disetahunkan digantikan tarif efektif rata-rata (TER) bulanan, tahun 2025–2026 menambah fasilitas PPh ditanggung pemerintah (DTP), dan parameter BPJS diperbarui setiap tahun. Di saat yang sama, setiap perusahaan memiliki kebijakan payroll sendiri. Dalam praktik, kedua jenis aturan itu ditanam bercampur di rumus *spreadsheet* atau kode program. Akibatnya perubahan regulasi menuntut perubahan rumus, kebijakan perusahaan dapat melanggar aturan wajib tanpa terdeteksi, dan angka di slip gaji tidak dapat ditelusuri ke pasal yang mendasarinya. Penelitian ini mengusulkan *knowledge-based system* untuk perhitungan PPh Pasal 21 tahun pajak 2023–2026 dengan tiga gagasan utama. Pertama, *knowledge base* deklaratif dua lapis, yaitu lapisan regulasi dan lapisan kebijakan perusahaan, dengan resolusi konflik berbasis asas *lex superior*, *lex specialis*, dan *lex posterior*. Kedua, versi waktu bitemporal (*valid time* dan *knowledge time*), sehingga satu *inference engine forward chaining* yang sama melayani dua rezim pajak dan regulasi yang berlaku surut. Ketiga, perlakuan deklaratif atas definisi melingkar *gross-up* sebagai *fixed point* dengan jaminan Knaster–Tarski. Setiap angka disertai *explanation trace* hingga aturan dan pasal sumbernya. Sistem dilengkapi demo Streamlit dan aplikasi web Laravel untuk admin *finance* yang memanggil *engine* lewat jembatan JSON tanpa menyalin rumus pajak. Karena penelitian ini tidak melibatkan *domain expert* maupun data payroll riil, kebenaran ditetapkan lewat *oracle pyramid*: 45 contoh resmi regulasi dengan 1.213 nilai harapan dan protokol erratum, *differential testing* terhadap *baseline hard-coded* yang dibekukan sebelum aturan *knowledge base* ditulis, *metamorphic relation* yang sebagian besar diuji dengan *property-based testing*, dan *mutation testing* atas *knowledge base*. Seluruh perhitungan tunduk pada kontrak presisi tanpa *floating point*. Draf ini menyajikan rancangan sistem dan metodologi evaluasi dua belas eksperimen. Hasil eksperimen belum final dan akan dilaporkan terpisah.**

Kata Kunci*—*knowledge-based system*, *rules as code*, PPh Pasal 21, *forward chaining*, *bitemporal versioning*, *conflict resolution*, *metamorphic testing*, *explainability

I. Pendahuluan

PPh Pasal 21 dipotong setiap bulan oleh pemberi kerja atas penghasilan pegawainya, sehingga ketepatan perhitungannya menyentuh setiap slip gaji. Aturannya juga sering berubah. Sampai tahun pajak 2023, pemotongan bulanan memakai metode disetahunkan menurut PER-16/PJ/2016 [1]. Sejak 1 Januari 2024, PP 58/2023 [2] dan PMK 168/2023 [3] menggantinya dengan tarif efektif rata-rata (TER) bulanan, ditambah perhitungan ulang setahun di masa pajak terakhir. Tahun 2025 dan 2026 menambahkan fasilitas PPh 21 ditanggung pemerintah (DTP) bagi sektor tertentu, yang salah satunya ditetapkan sesudah masa pajak yang dicakupnya dimulai [4]–[6]. Parameter BPJS seperti batas upah Jaminan Pensiun juga diperbarui setiap tahun. Di atas semua itu, setiap pemberi kerja memiliki kebijakan payroll sendiri, misalnya prorata kehadiran, kenaikan gaji di tengah bulan, cara menghitung THR, dan komponen tunjangan. Kebijakan itu menentukan berapa penghasilan bruto yang kemudian dikenai pajak.

Dalam praktik, aturan pemerintah dan kebijakan perusahaan ditanam bercampur di rumus *spreadsheet* atau kode program. Audit awal atas *spreadsheet* payroll sebuah perusahaan (Subbab III-C) menemukan tiga masalah. Konstanta regulasi tertanam di 254 sel rumus. Kebijakan perusahaan mengklasifikasikan lembur sebagai penghasilan tidak teratur, padahal regulasi menetapkannya sebagai penghasilan teratur. THR dipajaki pada masa yang tidak sesuai dengan saat terutangnya. Survei atas tujuh implementasi PPh 21 *open source*, berupa pustaka dan aplikasi, juga menemukan kesalahan terverifikasi di setiap implementasi. Contohnya lapisan tarif yang diperlakukan sebagai lebar, bukan batas kumulatif; premi BPJS pemberi kerja yang tidak masuk bruto; dan lebih bayar Desember yang dipotong menjadi nol. Temuan serupa dilaporkan pada implementasi resmi di negara lain. Formalisasi perhitungan tunjangan keluarga Prancis dengan bahasa Catala menemukan *bug* pada implementasi resminya [7], dan mesin pajak penghasilan Prancis sulit dipelihara karena aturannya bergantung pada bahasa dan *compiler* warisan [8].

Masalah ini dapat dirumuskan dalam empat butir:

1) Setiap perubahan regulasi menuntut perubahan rumus, dan

tidak ada jaminan bahwa perubahan itu tidak menggeser hasil tahun lain.
2) Batas antara aturan wajib pemerintah dan kebijakan perusahaan tidak eksplisit, sehingga pelanggaran aturan wajib tidak terdeteksi.
3) Angka di slip gaji tidak dapat ditelusuri ke pasal yang mendasarinya, sehingga sulit diaudit.
4) Definisi melingkar seperti *gross-up*, ketika tunjangan pajak sama dengan PPh atas penghasilan yang memuat tunjangan itu sendiri, diselesaikan dengan rumus turunan manual yang tidak lagi berlaku ketika struktur tarif berubah ke TER.

Menguji kebenaran perangkat lunak pajak pun sulit, karena spesifikasi eksplisit dan *test oracle* umumnya tidak tersedia [9], [10].

Pendekatan *rules as code* memisahkan aturan dari kode dengan menuliskan peraturan sebagai spesifikasi yang dapat dieksekusi [7], [11], [12]. Representasi formal pengetahuan pajak juga telah dipakai untuk payroll dan akuntansi pajak [13], [14], dan di Indonesia telah dibangun berbagai aplikasi perhitungan PPh 21 [15]–[17]. Namun, sejauh penelusuran penulis, belum ada pendekatan untuk PPh 21 yang sekaligus (a) memisahkan lapisan regulasi dari lapisan kebijakan perusahaan dengan resolusi konflik berbasis sifat aturan, (b) memvalidasi kedua rezim TER dan pra-TER dalam satu *knowledge base* berversi waktu, dan (c) menetapkan kebenaran sampai rupiah tanpa *domain expert*. Di sisi lain, *large language model* (LLM) mulai dipakai untuk penalaran hukum dan pajak, tetapi keandalannya masih terbatas, baik dalam penalaran undang-undang pajak [18], [19] maupun dalam menjawab pertanyaan faktual tentang hukum [20]. Karena itu, penelitian ini memakai *inference engine* deterministik untuk menghitung, dan LLM hanya dipakai di luar *engine* untuk menyusun rancangan aturan yang wajib disetujui manusia.

Penelitian ini mengusulkan *knowledge-based system* (KBS) untuk perhitungan PPh Pasal 21 pegawai tetap tahun pajak 2023–2026. Kontribusi yang diusulkan adalah sebagai berikut.

1) ***Knowledge base* dua lapis dengan resolusi konflik berbasis asas hukum.** Hubungan antara regulasi dan kebijakan pemberi kerja dimodelkan eksplisit melalui sifat aturan (`wajib`, `default`, `opsional`, `tafsir`) dan asas *lex superior*, *lex specialis*, dan *lex posterior*. Dengan begitu, pelanggaran aturan wajib terdeteksi otomatis.
2) **Versi waktu bitemporal.** *Valid time* memungkinkan satu *engine* menghitung rezim PER-16 dan rezim TER. *Knowledge time* memungkinkan pertanyaan "berapa yang dihitung pada saat itu" untuk regulasi yang berlaku surut. Adaptabilitas terhadap perubahan diukur dengan *impact precision*, *recall*, dan F1.
3) **Perlakuan deklaratif atas *gross-up* sebagai *fixed point*.** Keberadaan solusi dijamin teorema Knaster–Tarski, solusi ganda dideteksi lewat *least fixed point* dan *greatest fixed point*, dan pilihan antar-solusi dibuat deterministik.
4) **Metodologi validasi tanpa *domain expert*.** Kebenaran ditetapkan lewat *oracle pyramid*: contoh resmi dengan protokol erratum, *differential testing* terhadap *baseline* yang dibekukan lebih dulu, *metamorphic* dan *property-based testing*, serta *mutation testing*. Semuanya dijalankan di bawah kontrak presisi tanpa *floating point*.
5) **Artefak terbuka.** Artefak yang dirilis meliputi *knowledge base* PPh 21 2023–2026, *test case* resmi terstruktur beserta katalog erratum, kalkulator pembanding tanpa KB, demo Streamlit, dan aplikasi web Laravel untuk admin *finance*.

Pembahasan selanjutnya disusun sebagai berikut. Bagian II membahas penelitian terkait. Bagian III menguraikan metodologi penelitian: domain, data, representasi *knowledge base*, *inference engine*, kontrak presisi, *baseline*, antarmuka dan aplikasi web, strategi validasi, serta desain eksperimen. Hasil eksperimen akan dilaporkan pada versi berikutnya.

II. Tinjauan Pustaka

Penelitian ini bersinggungan dengan lima kelompok pustaka: (A) *rules as code* dan formalisasi peraturan pajak; (B) *knowledge-based system* untuk pajak dan payroll; (C) resolusi konflik norma dan representasi temporal peraturan; (D) pengujian perangkat lunak pajak dan masalah *test oracle*; serta (E) LLM untuk penalaran hukum, fasilitas penjelasan, dan pengawasan manusia.

A. Rules as Code dan Formalisasi Peraturan Pajak

Gerakan *rules as code* menuliskan peraturan sebagai spesifikasi yang dapat dieksekusi, sehingga aturan terpisah dari kode aplikasi. Catala [7] adalah bahasa pemrograman yang dirancang untuk menerjemahkan teks undang-undang secara sistematis, termasuk pola "kasus umum dan pengecualian" yang dimodelkan dengan *default logic*. Evaluasinya pada pajak penghasilan federal Amerika Serikat dan tunjangan keluarga Prancis menemukan *bug* pada implementasi resmi tunjangan tersebut. Huttner dan Merigoux [21] menegaskan bahwa *legal expert system* yang menghitung pajak berisiko hukum bila menyimpang dari undang-undang, dan bahwa proses pengembangan perangkat lunak konvensional sulit mengikuti arus perubahan peraturan. Mlang [8] dirancang untuk menggantikan *compiler* warisan mesin pajak penghasilan Prancis yang makin sulit dipelihara setiap kali undang-undang berubah. Mlang divalidasi terhadap *test suite* internal otoritas pajak, yang kemudian memulai peralihan resmi ke Mlang. Penelitian lanjutan dalam ekosistem Catala memformalkan aritmetika tanggal dan mendeteksi ambiguitasnya secara statis [22], serta membangkitkan *test case* dengan *concolic execution* untuk program pajak dan tunjangan [23].

Di luar Catala, DataLex merepresentasikan peraturan sebagai himpunan proposisi dalam bahasa aturan semi-alami yang dijalankan oleh *inference engine* beserta penjelasannya [11]. Penulisnya juga merumuskan sifat *explainability* yang diperlukan *rules as code*, antara lain *traceability* dan tautan ke sumber hukum [24]. Logical English menawarkan *controlled natural language* yang dapat dieksekusi sebagai program logika [12]. Meessen [25] membandingkan Catala dengan Regelspraak milik otoritas pajak Belanda dan menunjukkan bahwa banyak keputusan yang memengaruhi normativitas justru dibuat di dalam komponen teknis. Witt dkk. [26] membedakan validasi teknis (kode berjalan sesuai

konvensi) dari *legal alignment* (kode bermakna sama dengan undang-undang). Pembedaan ini dipakai di penelitian ini untuk menyatakan batas klaim secara jujur, karena evaluasinya tidak melibatkan pakar hukum.

Penelitian ini mengambil prinsip yang sama, yaitu memisahkan aturan dari kode. Bedanya ada pada tiga hal. Pertama, penelitian ini memisahkan secara eksplisit lapisan regulasi dari lapisan kebijakan perusahaan, beserta sifat aturan yang mengatur siapa boleh menimpa siapa. Kedua, satu *knowledge base* mencakup dua rezim pajak Indonesia sekaligus dengan versi waktu bitemporal. Ketiga, kebenaran sampai rupiah ditetapkan dengan metodologi validasi tanpa *domain expert*.

B. Knowledge-Based System untuk Pajak dan Payroll

Dumancic dkk. [13] menerjemahkan pengetahuan legislasi pajak dan jaminan sosial ke representasi formal yang cukup fleksibel untuk berbagai skenario perhitungan payroll. Pengetahuan itu lalu dikompilasi menjadi *constraint* agar *constraint solver* dapat mencari kebijakan payroll yang optimal, dan hasilnya dipakai di sebuah lembaga jaminan sosial. Penelitian ini menunjukkan bahwa pengetahuan pajak payroll layak dinyatakan secara deklaratif, terpisah dari kode. Fokusnya adalah optimasi kebijakan, bukan pemisahan lapisan regulasi dan kebijakan perusahaan maupun validasi kebenaran per rupiah. Al-Hawari dkk. [27] membangun sistem manajemen payroll dengan otomatisasi *rule-based* yang dilengkapi *chatbot retrieval-augmented generation*. Pada akuntansi pajak, Makowski dan Trąbka [14] memakai *Decision Model and Notation* (DMN) dengan dukungan AI generatif untuk merumuskan aturan keputusan pajak di kantor akuntan Swiss, dan melaporkan peningkatan *auditability* serta pemakaian ulang aturan.

Di Indonesia, penelitian tentang perhitungan PPh 21 umumnya membangun sistem informasi prosedural yang rumus pajaknya ditanam di aplikasi. Ramadhani dkk. [15] merancang aplikasi web PHP/MySQL untuk metode *gross-up*. Yulistyawati dkk. [16] merancang sistem informasi penggajian yang memadukan komponen gaji perusahaan dengan perhitungan PPh 21. Setelah TER berlaku, Kusuma dkk. [17] merancang sistem terintegrasi berbasis *spreadsheet* yang memuat tabel TER, rekapitulasi tahunan, dan keluaran bukti potong 1721-A1. Sistem itu dimotivasi oleh perbedaan antara perhitungan internal perusahaan dan perhitungan konsultan pajak. Sherra dan Fitriani [28] menunjukkan bahwa penghasilan tidak tetap seperti bonus dan THR membuat pemotongan TER bulanan menyimpang dari kewajiban setahun, sehingga terjadi lebih bayar. Mereka menyarankan agar penghasilan tidak teratur dialokasikan ke masa pajak terakhir dan lebih bayar dikompensasikan.

Sejauh penelusuran penulis atas artikel bereputasi sejak 2021, belum ditemukan sistem PPh 21 di Indonesia yang memisahkan aturan pemerintah dari kebijakan perusahaan sebagai dua lapis pengetahuan dengan resolusi konflik, menyertakan jejak penjelasan per angka, atau memvalidasi hasilnya terhadap contoh resmi sampai rupiah.

C. Resolusi Konflik Norma dan Representasi Temporal

Peraturan dan kebijakan dapat saling bertentangan, sehingga *rule-based system* memerlukan mekanisme untuk memilih norma mana yang berlaku. Robaldo dkk. [29] membandingkan beberapa *automated reasoner* untuk *compliance checking* pada norma yang saling bertentangan dan norma kompensasi, dan menyimpulkan bahwa keunggulan masing-masing *reasoner* belum terintegrasi dalam satu kerangka standar. Kharraz dkk. [30] mendefinisikan konflik pada logika normatif berwaktu melalui *satisfiability* dan menyediakan prosedur berbasis SMT untuk mendeteksinya. Khoja dkk. [31] memakai *solver* SMT untuk membuktikan adanya klausul kontrak yang saling bertentangan. Pada Catala, pola "kasus umum dan pengecualian" yang dimodelkan dengan *default logic* pada dasarnya mewujudkan *lex specialis* [7]. Dalam penelusuran penulis, pustaka sejak 2021 umumnya memperlakukan asas preferensi antarnorma sebagai relasi prioritas generik. Belum ditemukan formalisasi yang secara khusus membedakan aturan wajib pemerintah dari kebijakan pemberi kerja dalam perhitungan payroll.

Dimensi waktu sama pentingnya. Moguillansky dkk. [32] membedakan saat suatu ketentuan masuk ke sistem hukum, saat ketentuan itu berlaku, dan saat ia menimbulkan akibat hukum, termasuk akibat yang berlaku surut, lalu merumuskan operator revisi yang menjaga konsistensi berbasis waktu. De Martim [33] memodelkan peraturan sebagai karya yang memiliki ekspresi berversi agar jawaban tidak anakronistik. Olivieri dkk. [34] menunjukkan bahwa merevisi teori *defeasible logic* dengan menghapus aturan seminimal mungkin adalah masalah yang sulit secara komputasi.

Untuk kualitas basis aturan, Boujelben dan Amous [35] mendeteksi dan menyelesaikan anomali pada *rule base* yang terus diperbarui dengan memanfaatkan relasi dependensi antaraturan. Fungwacharakorn dkk. [36] menelusuri konsekuensi yang janggal dari representasi undang-undang kembali ke kondisi aturan penyebabnya.

Penelitian ini memakai gagasan-gagasan tersebut dalam bentuk yang deterministik dan dapat diaudit, dengan empat unsur. Pertama, asas *lex superior*, *lex specialis*, dan *lex posterior* diterapkan sebagai urutan resolusi terstratifikasi berdasarkan sifat aturan. Kedua, aturan berversi waktu bitemporal, yaitu *valid time* untuk masa berlaku dan *knowledge time* untuk regulasi yang berlaku surut. Ketiga, pencabutan aturan dinyatakan secara eksplisit lewat atribut `menggantikan` sebagai ganti revisi teori yang sulit dihitung. Keempat, verifikasi statis atas anomali dilakukan berbasis graf dependensi.

D. Pengujian Perangkat Lunak Pajak dan Masalah Test Oracle

Perangkat lunak pajak sulit diuji karena spesifikasi eksplisit dan *test oracle* umumnya tidak tersedia. Tizpaz-Niari dkk. [9] menjawab masalah ini pada perangkat lunak pajak penghasilan Amerika Serikat dengan *metamorphic testing*. *Metamorphic relation*-nya diturunkan dari publikasi resmi otoritas pajak bersama pakar pajak, dan pendekatan ini menemukan *bug*, mulai dari perilaku tidak robust pada kasus tepi (misalnya

pajak terutang mendekati nol) sampai syarat kelayakan yang hilang pada versi perangkat lunak yang diperbarui. Menyusun *metamorphic relation* dari dokumen pajak adalah pekerjaan manual yang mahal. Upaya mengotomatisasinya dengan *in-context learning* LLM menunjukkan potensi sekaligus keterbatasan [37], dan pendekatan *agentic* terbaru memperluasnya menjadi *higher-order metamorphic relation* sambil tetap menyatakan masalah *oracle* sebagai tantangan utama [10]. Survei terbaru menempatkan *metamorphic testing* sebagai teknik arus utama untuk masalah *oracle* dan merangkum cara menurunkan relasinya [38].

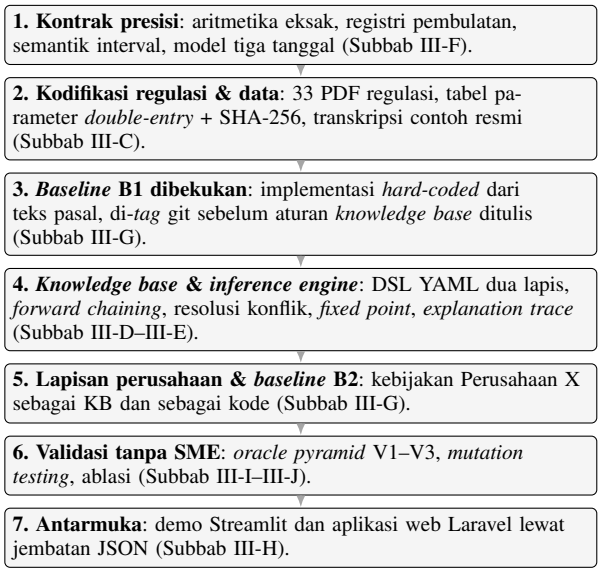
Property-based testing memeriksa spesifikasi yang dapat dieksekusi pada banyak input acak. Studi di perusahaan teknologi keuangan menunjukkan bahwa teknik ini menaikkan keyakinan atas kode yang kompleks melebihi pengujian konvensional, termasuk lewat pola *differential testing* terhadap implementasi rujukan [39]. Studi empiris pada proyek Python yang memakai pustaka Hypothesis menemukan bahwa satu *property-based test* rata-rata membunuh sekitar lima puluh kali lebih banyak mutan daripada satu *unit test* [40]. *Mutation analysis* sendiri dipandang sebagai salah satu kriteria kecukupan uji yang terkuat [41], dan analisis terhadap hampir 15 juta mutan menunjukkan bahwa mutan berkaitan dengan *fault* nyata [42].

Penelitian ini memakai teknik-teknik tersebut, tetapi menyusunnya sebagai *oracle pyramid* khusus untuk perhitungan pajak Indonesia. Puncaknya adalah contoh resmi regulasi dengan protokol erratum, diikuti *differential testing* terhadap *baseline* yang dibekukan lebih dulu, *metamorphic* dan *property-based testing*, lalu *mutation testing* yang diterapkan pada *knowledge base*, bukan pada kode. Semuanya dijalankan di bawah kontrak presisi dengan kriteria nol selisih rupiah yang tidak terjelaskan.

E. LLM, Penjelasan, dan Pengawasan Manusia

LLM mulai dipakai untuk penalaran hukum dan pajak, tetapi bukti yang ada menunjukkan keterbatasan serius untuk tugas yang menuntut ketepatan. Blair-Stanek dkk. [18] menemukan bahwa GPT-3 membuat kesalahan yang jelas pada penalaran undang-undang pajak, bahkan pada undang-undang sintetis sederhana yang tidak mungkin dilihatnya saat pelatihan. Nay dkk. [19] melaporkan bahwa akurasi LLM dalam menerapkan hukum pajak meningkat dari satu generasi model ke generasi berikutnya, tetapi belum mencapai tingkat ahli. Dahl dkk. [20] menunjukkan bahwa LLM publik berhalusinasi pada 58% sampai 88% pertanyaan faktual yang dapat diverifikasi tentang putusan pengadilan federal Amerika Serikat, tergantung modelnya. Alat riset hukum komersial berbasis *retrieval-augmented generation* pun masih berhalusinasi pada 17–33% kasus [43].

Di sisi lain, LLM dapat membantu tahap yang paling mahal dalam membangun sistem simbolik, yaitu mengubah teks peraturan menjadi representasi terstruktur. Pada studi Janatian dkk. [44], sekitar 60% struktur yang dihasilkan LLM dinilai setara atau lebih baik daripada hasil manual. Artinya sisanya tetap memerlukan tinjauan.



Gambar 1. Urutan tahapan penelitian. Urutan 2→3→4 menjaga agar B1 tidak dipengaruhi isi *knowledge base*.

Untuk penjelasan, metode penjelasan yang berpusat pada pengguna bagi sistem keputusan *rule-based* telah didemonstrasikan pada skenario nyata di otoritas pajak Belanda [45]. Sebaliknya, teknik XAI *post-hoc* pada model *machine learning* di bidang pajak dinilai belum memenuhi kewajiban hukum untuk memberikan alasan keputusan [46]. Pengawasan manusia diwajibkan untuk sistem AI berisiko tinggi [47], tetapi pengawasan manusia saja dapat memberi rasa aman yang palsu bila tidak didukung mekanisme lain [48].

Temuan-temuan ini mendasari pilihan desain penelitian ini. Perhitungan dijalankan oleh *inference engine* deterministik yang penjelasannya diturunkan langsung dari aturan dan pasal, bukan oleh LLM. LLM hanya dipakai di luar *engine* untuk menyusun *rancangan* aturan baru. Rancangan itu wajib melewati validasi otomatis oleh *engine* (skema, verifikasi statis, simulasi) *dan* persetujuan manusia sebelum berlaku.

F. Posisi Penelitian

Tabel I merangkum perbedaan penelitian ini dengan karya yang paling dekat. Tanda "–" berarti aspek tersebut tidak dilaporkan sebagai fokus atau kontribusi dalam publikasi yang dirujuk.

III. Metodologi Penelitian

Penelitian ini mengikuti pendekatan rekayasa artefak: sebuah *knowledge-based system* (KBS) dirancang, dibangun, lalu diuji terhadap pertanyaan penelitian yang dapat difalsifikasi. Gambar 1 merangkum urutan kerjanya. Urutan itu sengaja dibuat untuk menjaga independensi *oracle*: kontrak presisi numerik ditetapkan lebih dulu, kemudian kodifikasi regulasi dan verifikasi tabel, lalu *baseline hard-coded* B1 ditulis dan dibekukan *sebelum* berkas aturan *knowledge base* pertama dibuat. Setelah itu baru *knowledge base*, *inference engine*, lapisan perusahaan, perangkat validasi, dan antarmuka dibangun.

Tabel I
POSISI PENELITIAN TERHADAP KARYA TERKAIT

Karya	Domain	Representasi aturan	Regulasi vs kebijakan pemberi kerja	Versi waktu	Penjelasan	Validasi kebenaran
Catala [7], [21]	pajak dan tunjangan (Prancis, AS)	DSL dengan *default logic*	–	–	kode berdampingan dengan teks hukum	langkah inti *compiler* dibuktikan dengan F*; menemukan *bug* implementasi resmi
DataLex [11], [24]	legislasi umum (Australia)	bahasa aturan semi-alami + *inference engine*	–	–	*traceability*, tautan ke sumber hukum	–
Dumancic dkk. [13]	payroll (Belgia)	representasi formal → *constraint*	– (optimasi kebijakan)	–	–	dipakai lembaga jaminan sosial
Makowski & Trąbka [14]	akuntansi pajak (Swiss)	DMN + AI generatif	–	–	*auditability*	studi kasus deskriptif
Tizpaz-Niari dkk. [9]	pajak penghasilan (AS)	– (objek uji)	–	–	–	*metamorphic testing*
Sistem PPh 21 Indonesia [15]–[17]	PPh 21	aplikasi prosedural / *spreadsheet*	–	–	–	–
Penelitian ini	PPh 21 2023–2026	DSL YAML + *forward chaining*; *gross-up* sebagai *fixed point*	dua lapis; resolusi berbasis sifat aturan dan asas hukum	*valid time* + *knowledge time*	jejak aturan dan pasal per angka; rentang tafsir	*oracle pyramid*, *mutation testing*, kontrak presisi; target nol selisih rupiah tak terjelaskan

A. Pertanyaan Penelitian dan Hipotesis

Lima pertanyaan penelitian (RQ) dirumuskan bersama hipotesis yang dapat dibantah oleh satu contoh tandingan (Tabel II). Klaim kebenaran *tidak* memakai uji signifikansi statistik. Perhitungan pajak bersifat deterministik, sehingga satu tupel yang salah sudah cukup untuk menggugurkan klaim. Persentase dari data sintetis juga bergantung pada generator, sehingga statistik hanya dipakai secara deskriptif.

Hipotesis pada Tabel II disalin apa adanya dari rencana penelitian yang ditulis sebelum implementasi. Hipotesis itu tidak direvisi setelah uji coba berjalan. Selama pengembangan, sebagian besar eksperimen sudah dijalankan dalam bentuk uji coba (*pilot*) untuk menguji perangkatnya. Hasil uji coba itu belum final dan tidak dilaporkan di draf ini. Hipotesis yang nantinya terbantah akan dilaporkan sebagai terbantah, bukan dirumuskan ulang.

B. Domain dan Ruang Lingkup

Objek penelitian adalah pemotongan PPh Pasal 21 atas pegawai tetap, termasuk pegawai yang masuk atau keluar di tengah tahun, untuk tahun pajak 2023–2026. Rentang ini sengaja mencakup peralihan rezim. Tahun 2023 masih memakai metode disetahunkan menurut PER-16/PJ/2016 [1]: penghasilan sebulan disetahunkan, dikenai tarif Pasal 17 [49], lalu dibagi kembali per bulan. Sejak 1 Januari 2024 berlaku tarif efektif rata-rata (TER) bulanan menurut PP 58/2023 [2] dan PMK 168/2023 [3]. Pada rezim TER, PPh Januari–November dihitung dari tarif TER kategori A, B, atau C dikali bruto bulanan, lalu masa pajak terakhir menghitung ulang setahun dengan tarif Pasal 17. Tahun 2025–2026 menambahkan fasilitas PPh 21 ditanggung pemerintah (DTP) untuk sektor tertentu [4]–[6]. PTKP mengikuti PMK 101/2016 [50].

Komponen penghasilan yang dicakup adalah gaji, tunjangan tetap dan prorata, lembur, komisi, insentif, THR, bonus, premi BPJS yang dibayar pemberi kerja (JKK, JKM, Kesehatan), natura sederhana, dan tunjangan pajak. Pengurangnya adalah biaya jabatan, iuran pensiun/JHT/JP yang dibayar pegawai, dan zakat melalui pemberi kerja. Parameter BPJS mengikuti PP 44/2015, PP 45/2015, PP 46/2015, dan Perpres 82/2018 beserta perubahannya, termasuk batas upah JP yang diperbarui setiap 1 Maret. Metode yang dicakup adalah *gross*, *gross-up*, dan PPh ditanggung pemberi kerja.

Di luar lingkup penelitian: pegawai tidak tetap, bukan pegawai, dan pensiunan; penghasilan dari lebih dari satu pemberi kerja; pesangon (PPh final); pelaporan e-Bupot/Coretax; *gross-up* pada rezim 2023; serta data pribadi riil.

C. Akuisisi Pengetahuan dan Data

Penelitian ini tidak melibatkan *subject matter expert* (SME) dan tidak memakai data payroll riil. Karena itu, sumber pengetahuan dan sumber kebenaran dibangun dari dokumen resmi serta data publik yang dapat diperiksa ulang.

1) Korpus regulasi dan kodifikasi: Sebanyak 33 berkas PDF regulasi dikumpulkan, mayoritas dari JDIH BPK dan situs DJP, dan setiap berkas diperiksa keutuhannya. Setiap ketentuan yang relevan dikodifikasi menjadi baris berisi kondisi, akibat, pengecualian, parameter, masa berlaku, pasal sumber, sumbu waktu, dan titik pembulatan. Ketentuan yang dapat dibaca lebih dari satu cara dicatat sebagai *interpretation variant* (*tafsir*), lengkap dengan varian bawaan dan alternatifnya.

Tabel II
PERTANYAAN PENELITIAN, HIPOTESIS, DAN EKSPERIMEN PENGUJINYA

RQ	Pertanyaan	Hipotesis	Diuji di
RQ1 Representasi	Dapatkah aturan PPh 21 tahun 2023–2026 dan kebijakan perusahaan dinyatakan dalam *knowledge base* dua lapis berversi waktu tanpa kode khusus?	H1: $\geq$90% dari 20 kebijakan katalog dapat dinyatakan *native* tanpa perubahan *engine*.	E7
RQ2 Kebenaran	Apakah setiap angka, per masa dan per komponen antara, identik sampai rupiah dengan *oracle*, dan apakah setiap selisih dapat dijelaskan?	H2a: 0 selisih terhadap contoh resmi terkoreksi. H2b: 0 selisih tak terjelaskan pada data sintetis terhadap *oracle* teradjudikasi.	E1, E2, E8, E10–E12
RQ3 Adaptabilitas	Ketika regulasi atau kebijakan berubah, seberapa kecil dan seberapa tepat perubahan yang diperlukan?	H3a: semua skenario perubahan tertangani tanpa mengubah kode *engine*. H3b: *impact* F1 $= 1{,}0$. H3c: entri KB yang berubah jauh lebih sedikit daripada baris kode pada *baseline*.	E3
RQ4 Konflik	Dapatkah konflik antarlapisan dan kesalahan KB dideteksi secara otomatis?	H4: F1 deteksi $\geq 0{,}95$ pada mutan KB dan konflik berlabel.	E4
RQ5 *Gross-up*	Kapan persamaan *gross-up* di bawah TER memiliki solusi tunggal atau ganda?	H5a: solusi selalu ada. H5b: ada pita solusi ganda di setiap batas lapisan TER. H5c: di masa pajak terakhir solusinya tunggal.	E6
Ablasi	Apakah setiap fitur KB benar-benar diperlukan?	Mematikan satu fitur menghasilkan ≥ 1 kasus salah yang dapat ditunjukkan.	E5

2) Tabel parameter dengan double-entry: Tabel TER bulanan (125 lapisan: A = 44, B = 40, C = 41), tarif Pasal 17 (lapisan UU HPP dan pra-HPP), PTKP (12 status), parameter BPJS, dan daftar KLU penerima DTP (269 baris) disimpan sebagai CSV. Untuk menghindari *single point of failure*, tabel parameter diekstraksi dua kali secara independen dengan metode berbeda. Ekstraksi pertama mem-*parse* lapisan teks (sebagian hasil OCR penerbit) dan memeriksa angka secara visual. Ekstraksi kedua membaca gambar halaman yang dirender, tanpa melihat hasil ekstraksi pertama. Kedua hasil dibandingkan sel demi sel: 512 sel pada TER bulanan dan harian, Pasal 17 rezim UU HPP, PTKP, kategori TER, biaya jabatan, dan BPJS. Baris Pasal 17 pra-HPP dan daftar KLU belum melewati *double-entry*; status ini tercatat di *manifest*, dan daftar KLU masih perlu dicek ke gambar halaman. Tabel final dikunci dengan *hash* SHA-256 di *manifest* itu, dan *loader* menolak tabel yang *hash*-nya tidak cocok.

3) Contoh perhitungan resmi: Contoh perhitungan dalam lampiran PMK 168/2023, PP 58/2023, PER-16/PJ/2016, dan PMK DTP 2025–2026 ditranskripsi menjadi *test case* kanonik berformat JSON. Setiap kasus memuat rujukan halaman dan diverifikasi dua kali terhadap gambar halaman PDF. Hasilnya 45 kasus kanonik dengan 1.213 nilai harapan: 21 dari PER-16, 13 dari PMK 168, 1 dari PP 58, dan 10 dari PMK DTP. Transkripsi menemukan bahwa contoh resmi pun memuat salah cetak dan pasangan contoh yang saling bertentangan. Karena itu diterapkan *protokol erratum*:

1) Nilai harapan selalu disimpan sebagaimana tercetak.
2) Nilai yang salah cetak diberi `terkoreksi` yang diturunkan dari pasal berotoritas lebih tinggi, beserta alasannya (20 nilai).
3) Nilai yang bergantung pada tafsir diberi varian tafsir yang mereproduksinya (8 nilai).

Hierarki otoritas yang dipakai, dari tertinggi: undang-undang, peraturan pemerintah, batang tubuh PMK/PER, lampiran contoh PMK/PER, kalkulator dan FAQ resmi DJP, lalu sumber sekunder.

4) Studi kasus Perusahaan X: Sebuah *spreadsheet* payroll kantor (*workbook* Excel, tahun pajak 2023) dipakai sebagai studi kasus kebijakan perusahaan nyata dan dianonimkan sebagai "Perusahaan X". Hanya *sheet* kalkulator dengan data *dummy* yang dibaca; *sheet* berisi data pribadi tidak dibaca, tidak disalin, dan tidak dipublikasikan. Audit awal atas *spreadsheet* itu menemukan konstanta regulasi yang ditanam di 254 sel rumus, lembur yang diklasifikasikan sebagai penghasilan tidak teratur (bertentangan dengan PER-16 Pasal 1 angka 15), dan THR yang dipajaki pada masa yang keliru. Temuan ini menjadi motivasi lapisan perusahaan yang terpisah dan diawasi oleh aturan wajib. *Spreadsheet* tersebut diperlakukan sebagai *system under test*, bukan sebagai kebenaran.

5) Data sintetis: Dua jenis data sintetis dipakai.

- ***Test case* berstrata** (2.600 kasus, *seed* tetap) dibangkitkan dari delapan strata titik rawan: S1 acak, dengan gaji berdistribusi log-normal di sekitar median yang diasumsikan dan bobot status PTKP kasar (belum dikalibrasi ke data BPS); S2 bruto tepat di batas lapisan TER ($b-1$, b, $b+1$); S3 pita *gross-up*; S4 masuk/keluar di tengah tahun; S5 penghasilan tidak teratur besar; S6 profil identik lintas rezim 2023–2026; S7 bruto pecahan (prorata, kenaikan gaji di tengah bulan); dan S8 sumbu waktu (tanggal bayar berbeda dari periode kerja).
- **Pegawai sintetis berformat data HR** (8 profil). Gaji pokoknya dibaca langsung dari sel data publik, yaitu UMP Kemnaker 2023–2026 dan rata-rata upah bersih BPS Sakernas per provinsi dan sektor, dengan sumber yang dicatat per pegawai. Tanggal Idul Fitri untuk THR diambil dari SKB tiga menteri tentang hari libur nasional. Status PTKP serta tanggal masuk dan keluar adalah variasi skenario yang dinyatakan eksplisit sebagai sintetis.

6) Katalog kebijakan: Katalog 20 kebijakan perusahaan yang umum, KP-01 sampai KP-20, disusun dari regulasi ketenagakerjaan dan praktik payroll. Contohnya THR prorata per bulan sesuai Permenaker 6/2016, lembur per jam 1/173, dasar upah BPJS, prorata hari kalender, natura makan bersama,

```
- id: R24-03
  sifat: wajib
  berlaku: {mulai: 2024-01-01}
  lingkup: masa
  menghasilkan: _bruto_dasar
  maka: "komponen('teratur','bulan')
          + komponen('tidak_teratur','bulan')
          + komponen('premi_objek','bulan') + ..."
  pembulatan: BULAT-BRUTO-01
  tipe_hasil: rupiah
  sumber: "PMK 168/2023 Ps. 5(3), 8(2)a, ..."
```

Gambar 2. Contoh aturan lapisan regulasi dalam DSL YAML (disingkat; "`...`" menandai bagian yang dipotong).

zakat, dan DPLK. Katalog ini dipakai untuk mengukur ekspresivitas (RQ1). Kebijakan yang bertentangan dengan aturan wajib diberi label untuk mengukur deteksi konflik (RQ4).

D. Representasi Knowledge Base

1) Dua lapis pengetahuan: Knowledge base dibagi menjadi *lapisan regulasi* dan *lapisan perusahaan*. Lapisan regulasi memuat aturan pemerintah, tabel, dan parameter. Lapisan perusahaan memuat cara perusahaan menghitung komponen gaji dari data HR mentah, misalnya prorata hari kerja, kenaikan gaji di tengah bulan, tunjangan tetap dan prorata, premi dan iuran BPJS, THR, kompensasi, dan *take-home pay*.

Kedua lapis bertemu di *pemetaan komponen*. Setiap komponen perusahaan wajib dipetakan ke satu kategori pajak, yaitu `teratur`, `tidak_teratur`, `premi_objek`, `natura`, `iuran_pengurang`, `zakat`, `bukan_objek`, atau `tidak_diperhitungkan`. Di sisi lain, lapisan regulasi menyimpan *klasifikasi wajib* untuk 16 jenis komponen, misalnya lembur sebagai penghasilan teratur. Bila pemetaan perusahaan berbeda dari klasifikasi wajib, kategori regulasi yang dipakai dan sistem mengeluarkan peringatan `KONFLIK_WAJIB`. Bila regulasi tidak mengatur suatu jenis komponen, kategori perusahaan dipakai dengan peringatan `KLASIFIKASI_TIDAK_DIATUR`.

2) Bahasa aturan: Aturan ditulis dalam *domain-specific language* (DSL) berbasis YAML dan divalidasi dengan JSON Schema sebelum dimuat. Lapisan ditetapkan per berkas. Di dalam berkas, satu aturan menyatakan satu fakta yang dihasilkannya, beserta atribut berikut:

- `sifat` (`wajib`, `default`, `opsional`, atau `tafsir`), ditambah kelompok dan nama varian untuk aturan tafsir;
- masa berlaku `berlaku` (*valid time*) dan tanggal pencatatan `dicatat` (*knowledge time*);
- lingkup (`masa` atau `tahun`) dan sumbu waktu;
- kondisi `jika` dan ekspresi `maka`;
- tipe hasil, rujukan entri registri pembulatan, dan `sumber`, yaitu pasal beserta regulasinya;
- atribut opsional: prioritas, pencabutan tegas `menggantikan`, serta tanda dan batas *fixed point*.

Gambar 2 memperlihatkan salah satu aturan.

Ekspresi `jika` dan `maka` dibatasi pada bahasa ekspresi yang aman. Bahasa ini hanya menerima aritmetika, perbandingan, dan 33 fungsi bawaan, misalnya pencarian tabel TER dan Pasal 17, agregasi antar-masa, fungsi tanggal, pembacaan data HR, dan pembulatan terdaftar. Ekspresi divalidasi sebagai *abstract syntax tree* sebelum dievaluasi, dan tidak ada *escape hatch* ke kode bebas. Kebijakan yang tidak dapat dinyatakan dengan bahasa ini dihitung secara jujur sebagai "tidak bisa" pada metrik ekspresivitas.

Tabel III
ISI *knowledge base* DASAR

Unsur	Jumlah
Aturan lapisan regulasi (umum, TER, PER-16, DTP)	90
Aturan lapisan perusahaan (Perusahaan X)	32
Fakta turunan yang dihasilkan aturan	88
Parameter berversi (nama / versi)	14 / 18
Lapisan tabel TER bulanan (A/B/C)	125
Baris tarif Pasal 17 (UU HPP + pra-HPP)	9
Status PTKP	12
Baris KLU penerima DTP	269
Entri registri pembulatan (wajib/tafsir/kebijakan)	1/8/1
Klasifikasi wajib jenis komponen	16
Fungsi bawaan bahasa ekspresi	33

3) Isi knowledge base dasar: Tabel III merangkum isi *knowledge base* dasar, yaitu lapisan regulasi ditambah Perusahaan X. Jumlah aturan yang aktif berbeda per tahun karena masa berlakunya: 93 aturan pada 2023, 72 pada 2024, dan 82 pada 2025 maupun 2026.

4) Verifikasi statis: Sebelum dievaluasi, *knowledge base* melewati pemeriksaan berikut. Sebagian dijalankan saat KB dimuat, dan sebagian saat graf dependensi dibangun untuk tahun evaluasi.

- validasi skema;
- penolakan *float literal* dan kunci YAML ganda;
- kecocokan *hash* tabel;
- kelengkapan dan ketidaktumpangtindihan lapisan setiap versi tabel bertingkat;
- tumpang tindih masa berlaku antar-versi tabel Pasal 17;
- keunikan identitas aturan;
- konsistensi lingkup dan tipe hasil per fakta;
- kewajiban batas iterasi bagi aturan *fixed point*;
- keunikan varian bawaan per kelompok tafsir;
- keberadaan setiap fakta yang dirujuk pada tahun evaluasi;
- penolakan siklus yang tidak ditandai *fixed point*.

E. Inference Engine

Inference engine adalah program generik yang tidak memuat pengetahuan pajak. Ia membaca *knowledge base*, lalu mengevaluasi fakta dengan *forward chaining* terstratifikasi (Algoritma 1).

1) Seleksi aturan dan urutan evaluasi: Aturan dipilih menurut masa berlakunya terhadap tahun pajak. Bila diminta perhitungan "sebagaimana diketahui pada tanggal κ", aturan dengan tanggal `dicatat` sesudah κ juga dikeluarkan. Ketergantungan antar-fakta dibaca dari pohon ekspresi setiap aturan, termasuk rujukan tidak langsung lewat fungsi agregasi. Graf dependensi itu lalu dipecah menjadi *strongly connected components* (SCC) dengan algoritma Tarjan dan dievaluasi dalam urutan topologis. SCC yang bersiklus hanya diterima

Algoritma 1 Evaluasi satu pegawai untuk satu tahun pajak

Masukan: kasus k (tahun y, masa M, fakta dasar), KB, varian tafsir V, tanggal pengetahuan κ (opsional)

```
k ← TERAPKANTRANSAKSI(k)                          ▷ masa = saat terutang
VALIDASIINPUT(k)                                  ▷ tolak float, data kurang
R ← {r ∈ KB : y ∈ r.berlaku ∧ (κ = ⊥ ∨ r.dicatat ≤ κ) ∧ r sesuai V}
Bangun graf dependensi G atas fakta yang dihasilkan R
C ← TARJAN(G)                                     ▷ SCC dalam urutan topologis
untuk setiap C ∈ C lakukan
    jika C bersiklus maka
        SELESAIKANFIXEDPOINT(C)                   ▷ Subbab III-E3
    selain itu
        untuk setiap instance (a, m), m ∈ M atau tahunan lakukan
            K ← {r ∈ R : r menghasilkan a ∧ r.jika(m)}
            r* ← RESOLUSI(K)                      ▷ Subbab III-E2
            v ← BULATKAN(r*.maka(m), r*.pembulatan)
            simpan (a, m, v) dan jejak (r*, pasal, aturan ditolak)
        akhir for
    akhir jika
akhir for
kembalikan fakta per masa dan tahunan, jejak, peringatan
```

bila memuat aturan bertanda *fixed point*, misalnya tunjangan pajak pada metode *gross-up*. Siklus lain ditolak.

2) Resolusi konflik berbasis asas hukum: Untuk fakta a pada masa m, himpunan kandidat K berisi aturan yang menghasilkan a, berlaku, dan kondisinya terpenuhi. Sebelum resolusi, aturan yang dicabut secara tegas oleh aturan lain lewat atribut `menggantikan`, selama aturan pengganti berlaku, dikeluarkan dari K. Resolusi berjalan dalam dua tahap.

1) **Pemilihan kelompok (*lex superior*).** Bila ada aturan regulasi bersifat `wajib` atau `tafsir`, kelompok ini dipakai dan aturan perusahaan yang bersaing ditolak dengan peringatan `KONFLIK_WAJIB`. Bila tidak ada, aturan perusahaan dipakai dan mengalahkan aturan regulasi bersifat `default`/`opsional` (*override* yang sah). Bila keduanya tidak ada, aturan regulasi lainnya dipakai.
2) **Di dalam kelompok.** Urutannya: *lex specialis* (kondisi dengan konjungsi tingkat atas terbanyak), lalu *lex posterior* (tanggal mulai berlaku terbaru), lalu prioritas eksplisit.

Bila setelah itu masih tersisa lebih dari satu aturan dan nilainya berbeda, evaluasi dihentikan dengan galat `AMBIGU` alih-alih memilih secara diam-diam. Seluruh keputusan, termasuk aturan yang ditolak dan alasannya, dicatat di jejak.

3) Fixed point untuk gross-up: Pada metode *gross-up*, tunjangan pajak T sama dengan PPh atas bruto yang sudah memuat T itu sendiri:

$$T = f(T) = \mathrm{PPh}_{\mathbb{Z}}(B + T), \tag{1}$$

dengan B bruto sebelum tunjangan pajak dan $\mathrm{PPh}_{\mathbb{Z}}$ PPh dalam rupiah bulat yang sudah memuat pembulatan resmi. Fungsi f monoton tidak turun, dan aturan *fixed point* wajib menyatakan batas bawah dan batas atas iterasi yang memetakan selangnya ke dirinya sendiri. Dengan dua syarat itu, teorema *fixed point* Knaster–Tarski menjamin bahwa *least fixed point* dan *greatest fixed point* ada.

Engine menghitung keduanya dengan iterasi Kleene, masing-masing dari batas bawah dan dari batas atas, disertai pengaman monotonisitas. Bila kedua hasil berbeda, sistem mengeluarkan peringatan `GROSSUP_GANDA` beserta kedua solusinya, lalu memilih secara deterministik menurut varian tafsir `TITIK-TETAP-PILIH` (bawaan: terkecil). Dengan cara ini hasil tidak bergantung pada jalur iterasi. *Gross-up* bulanan (TER) dan *gross-up* masa pajak terakhir, yang merupakan persamaan tahunan, ditempatkan di SCC terpisah. Pada masa pajak terakhir, T boleh negatif bila terjadi lebih bayar.

4) Fasilitas penjelasan dan rentang tafsir: Setiap fakta turunan menyimpan aturan pemenang, lapisan dan sifatnya, pasal sumber, serta aturan yang ditolak beserta alasannya, misalnya *lex superior* atau *lex specialis*. Untuk PPh Pasal 17 setahun, *engine* juga mengeluarkan rincian per lapisan tarif (dasar kena pajak dan pajak tiap lapisan), sehingga perhitungan bergaya bukti potong 1721-A1 dapat ditampilkan tanpa rumus tambahan di antarmuka.

Untuk ketentuan berstatus tafsir, perhitungan diulang dengan setiap varian alternatif. Selisihnya terhadap varian bawaan dilaporkan sebagai *rentang tafsir* per angka, sehingga eksposur risikonya terlihat dan bukan hanya satu angka yang tampak pasti. Saat ini rentang tafsir tersedia lewat CLI dan API *engine*, tetapi belum ditampilkan di demo Streamlit maupun aplikasi web.

F. Kontrak Presisi Numerik dan Waktu

Dalam ranah keuangan, selisih Rp1 diperlakukan sebagai *bug*. Seluruh komponen sistem, yaitu *engine*, *knowledge base*, *baseline*, dan antarmuka, tunduk pada kontrak berikut.

1) **Aritmetika eksak.** Nilai uang bertipe bilangan bulat rupiah, sedangkan tarif dan nilai antara bertipe pecahan rasional eksak (`Fraction`) yang dibaca dari teks. Literal *floating point* di YAML ditolak *loader*. Pemanggilan `float()` dan `round()` dilarang di kode *engine* dan *baseline*, dan larangan ini ditegakkan oleh pemindai *abstract syntax tree* dalam *test suite*. Uji awal pada 44 tarif TER dan bruto kelipatan Rp1.000 dari Rp1.000 sampai Rp1.999.000 menunjukkan bahwa aritmetika *float* menghasilkan 700 selisih Rp1 dibanding aritmetika eksak bila hasilnya dibulatkan ke bawah.
2) **Registri pembulatan.** Setiap aturan bertipe rupiah wajib merujuk satu entri registri yang menyebut titik, satuan, mode, urutan, status (`wajib`, `tafsir`, atau `kebijakan`), dasar hukum, dan alternatifnya. Ada enam mode dengan semantik eksplisit untuk bilangan negatif: `bawah`, `atas`, `menuju_nol`, `setengah_atas`, `setengah_menjauhi_nol` (padanan `ROUND` di Excel), dan `setengah_genap`. Satu-satunya titik yang diatur tegas oleh regulasi adalah PKP yang dibulatkan ke bawah ke ribuan rupiah. Titik lain berstatus tafsir dan menjadi sasaran kalibrasi E10.
3) **Semantik interval.** Frasa "di atas a sampai dengan b" ditulis sebagai interval $(a, b]$. Setiap versi tabel bertingkat wajib menutup $[0, \infty)$ tanpa celah maupun tumpang tindih.
4) **Model tiga tanggal.** Setiap transaksi penghasilan membawa periode kerja, tanggal terutang, dan tanggal bayar. Masa pajak ditentukan oleh *saat terutang* $=$ $\min(\text{tanggal bayar}, \text{tanggal terutang})$ sesuai PMK

168/2023 Pasal 19 ayat (1) untuk tahun 2024 dan sesudahnya. Untuk tahun 2023 aturan yang sama diterapkan sebagai tafsir. Parameter BPJS mengikuti periode iuran.

5) **Validasi input.** Input yang tidak lengkap atau ambigu menghentikan perhitungan; sistem tidak mengisi nilai bawaan secara diam-diam. Nominal wajib bilangan bulat ≥ 0. Keluaran yang diminta lewat CLI dan jembatan aplikasi disertai metadata audit berupa versi *knowledge base*, *hash* tabel, dan varian tafsir non-bawaan yang diminta secara eksplisit. Varian bawaan yang terpakai belum dicatat satu per satu di metadata ini.

G. Baseline Pembanding

Tidak ada *leaderboard* untuk perhitungan PPh 21, sehingga pembanding yang dipakai adalah implementasi praktik. Semua pembanding berstatus *system under test*, bukan *oracle*.

- **B0**: *spreadsheet* Perusahaan X (tahun 2023), dengan aturan yang tertanam di sel rumus.
- **B1**: kalkulator PPh 21 *hard-coded* dalam Python, ditulis langsung dari teks pasal tanpa melihat *knowledge base* dan tunduk pada kontrak presisi yang sama. B1 diberi *tag* git sebelum berkas aturan regulasi pertama dibuat, sehingga urutan pembuatannya dapat dibuktikan dari riwayat *commit*. Registri pembulatan dan *manifest* tabel (tahap 1–2) sudah ada sebelum *tag* itu dan dipakai bersama oleh B1 dan *knowledge base*. B1 menjadi versi independen kedua untuk *differential testing*.
- **B2**: kalkulator payroll *tanpa knowledge base*. Kebijakan Perusahaan X (prorata, kenaikan gaji, BPJS, THR, kompensasi, *take-home pay*) beserta konstantanya ditanam di kode, sedangkan pajaknya dihitung oleh B1 tanpa perubahan. B2 diuji agar tidak mengimpor *engine* maupun membaca berkas *knowledge base*. B2 adalah pembanding kontrafaktual yang adil: fiturnya sama dengan versi KB, tetapi mekanismenya berbeda.
- **B3**: implementasi *open source*, berupa pustaka dan aplikasi (survei 35 kandidat, 7 di antaranya diuji), dan kalkulator daring, termasuk kalkulator resmi DJP yang dipakai secara manual sebagai penengah kalibrasi tafsir (E10).

H. Antarmuka dan Aplikasi Web

Sistem dilengkapi dua antarmuka. Keduanya *tidak* memuat logika pajak: setiap angka yang ditampilkan berasal dari *engine* (Gambar 3). Dengan begitu antarmuka mewarisi bukti verifikasi *engine* untuk *knowledge base* dasar dan tidak menjadi sumber galat baru.

1) Demo Streamlit: Demo Streamlit dipakai untuk riset dan presentasi. Tab *Kalkulator* menyajikan dua kalkulator dengan tampilan identik tetapi mekanisme berbeda, yaitu dengan KB (*engine*) dan tanpa KB (B2+B1). Isiannya hanya data yang biasa diisi admin *finance*, dengan nilai bawaan dari dataset: Karyawan A, delapan pegawai sintetis, dan 45 contoh resmi. Keluarannya meliputi slip gaji per bulan, perhitungan setahun bergaya 1721-A1, dan tabel 12 bulan. Setiap angka disertai kolom *dasar*: aturan dan pasal pada versi KB, atau fungsi dan baris kode pada versi tanpa KB.

Setiap hasil juga diberi *cek silang* otomatis terhadap kalkulator satunya. Untuk contoh resmi yang belum diubah, hasil dicocokkan dengan nilai harapan dokumen memakai protokol V1. Tab *Knowledge Base* menyajikan pengetahuan per topik, yaitu nilai yang berlaku per tahun, pasal sumber, label lapisan, dan konflik klasifikasi, dalam bahasa yang dapat dipahami pengguna non-teknis.

2) Aplikasi web Laravel: Aplikasi web multi-pengguna untuk admin *finance* dibangun dengan Laravel (PHP $\geq$ 8.3). Basis datanya SQLite atau PostgreSQL, dan strukturnya terdiri atas modul *Pegawai*, *Payroll*, *Dashboard*, *Basis pengetahuan*, dan *Mesin*. Aplikasi menyusun kasus kanonik dari basis data, termasuk data HR per tahun pajak serta kehadiran dan penghasilan variabel per bulan. Kasus itu dikirim ke *engine* lewat jembatan JSON: proses Python yang menerima satu permintaan di *stdin* dan menjawab di *stdout*, dengan perintah seperti `hitung`, `masukan`, `validasi`, `periksa`, dan `usulkan`.

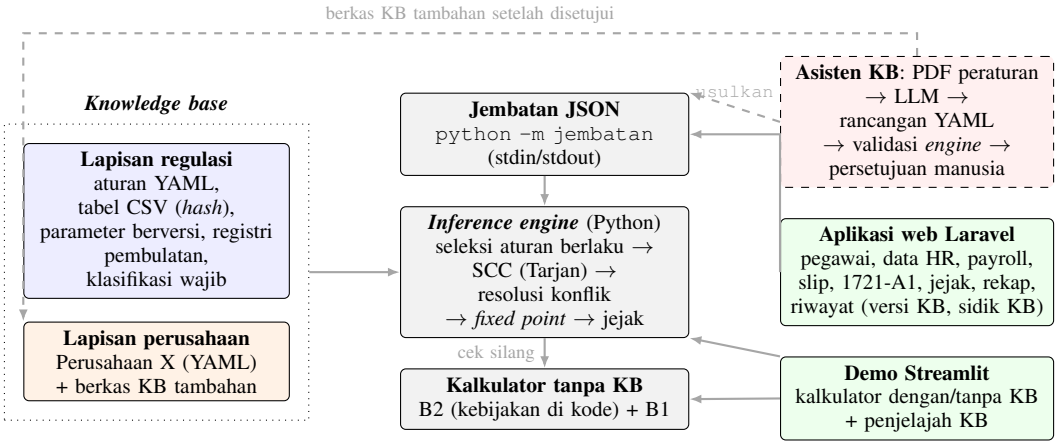
Hasil *engine* disimpan apa adanya sebagai riwayat yang tidak pernah ditimpa. Setiap entri riwayat memuat kasus yang dikirim, keluaran *engine*, versi *knowledge base*, sidik berkas KB tambahan, dan identitas pengguna. Bila data HR atau berkas KB berubah sesudah perhitungan, hasil lama ditandai usang. Untuk jumlah pegawai yang besar, perhitungan massal dijalankan per *batch* dengan kunci per tahun pajak. Keluaran yang ditampilkan meliputi slip, perhitungan 1721-A1 dengan rincian Pasal 17, *inference trace* lengkap (termasuk aturan yang ditolak atau dicabut), peringatan konflik, dan cek silang terhadap kalkulator tanpa KB. Cek silang ini dilewati bila perhitungan memakai berkas KB tambahan (Subbab berikut), karena kalkulator tanpa KB tidak mengenal aturan baru. Sebuah uji integrasi memastikan bahwa kasus yang disusun dari basis data untuk sembilan pegawai contoh identik dengan kasus di dataset.

3) Berkas KB tambahan dan asisten KB: Aplikasi web juga menyediakan jalur operasional bagi aturan baru: peraturan pemerintah atau perusahaan dapat dimasukkan sebagai *berkas KB tambahan* tanpa mengubah kode. Satu berkas dapat berisi:

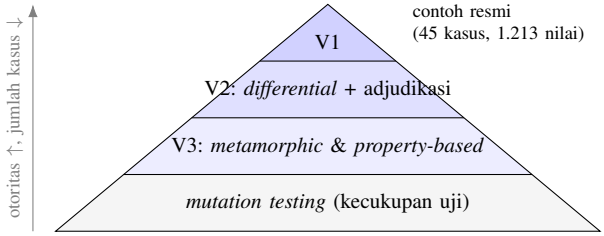
- aturan baru, termasuk pencabutan tegas aturan lama lewat `menggantikan`;
- komponen gaji baru beserta kategori pajaknya;
- amandemen parameter dan klasifikasi wajib yang berversi menurut tanggal mulai berlaku;
- deklarasi `masukan`, yaitu isian baru yang diminta dari pengguna; aplikasi web membangun formulirnya secara otomatis dari deklarasi ini.

Tabel resmi (TER, Pasal 17, PTKP, KLU) tidak dapat diubah lewat jalur ini, karena perubahannya tetap harus melewati *double-entry* dan *manifest*.

Rancangan berkas KB tambahan dapat disusun oleh *asisten KB* dari PDF peraturan menggunakan *large language model* (LLM) dengan *structured output*. Asisten ini berada di luar *engine*. LLM tidak menghitung pajak, tidak dipakai sebagai *oracle* maupun *baseline*, dan tidak dapat mengubah tabel resmi. Rancangan baru berlaku setelah melewati dua gerbang:



Gambar 3. Arsitektur sistem. *Knowledge base* dibaca oleh *inference engine* generik. Aplikasi web memanggil *engine* lewat jembatan JSON dan tidak menghitung pajak sendiri. Kalkulator tanpa KB (B2+B1) dipakai sebagai cek silang. Jalur bergaris putus-putus (asisten KB berbasis LLM) hanya menghasilkan *rancangan* aturan yang wajib divalidasi *engine* dan disetujui manusia, dan berada di luar evaluasi penelitian ini.



Gambar 4. *Oracle pyramid* yang dipakai untuk menetapkan kebenaran tanpa *domain expert*.

1) **Validasi *engine***: skema, verifikasi statis, kewajiban mendeklarasikan setiap isian baru, dan simulasi sebelum–sesudah pada dua pegawai contoh. Simulasi ini juga melaporkan aturan yang tidak pernah terpilih sebagai *belum teruji*.
2) **Persetujuan manusia** di aplikasi web, dengan kutipan dan nomor halaman PDF untuk setiap aturan.

Validasi *engine* memeriksa bentuk dan konsistensi rancangan, bukan kesesuaiannya dengan isi dokumen. Pencocokan itu adalah tugas peninjau manusia.

Dalam penelitian ini, jalur berkas KB tambahan dan asisten KB diperlakukan sebagai *perluasan yang belum dievaluasi*. Mutu ekstraksi LLM terhadap dokumen nyata belum diukur, dan jalur ini belum diuji dengan protokol RQ3 (Subbab III-J). Karena itu tidak ada klaim kuantitatif tentang asisten KB yang diajukan.

I. Strategi Validasi tanpa SME: Oracle Pyramid

Karena tidak ada SME dan kalkulator yang sudah ada tidak dipercaya, kebenaran ditetapkan berlapis. Semakin tinggi lapisannya, semakin tinggi otoritasnya, tetapi semakin sedikit kasusnya (Gambar 4).

1) V1: contoh resmi: Setiap nilai harapan dibandingkan dengan keluaran sistem pada tingkat tupel (kasus, masa, fakta). Pemeriksaan mencakup komponen antara, seperti bruto, kategori dan tarif TER, biaya jabatan, PKP, dan PPh masa terakhir, bukan hanya total tahunan. Pemeriksaan total saja tidak memadai, karena masa pajak terakhir selalu menyamakan total setahun sehingga galat bulanan bisa tertutup. Nilai harapan dibandingkan dengan protokol erratum (Subbab III-C).

V1 hampir tidak menguji pembulatan. Dari 170 perhitungan TER bulanan di contoh resmi, hanya satu yang menghasilkan pecahan rupiah. Pembulatan karena itu diuji tersendiri lewat strata S7 dan kalibrasi E10.

2) V2: differential testing dan adjudikasi: Untuk setiap kasus sintetis, keluaran KB dan B1 dibandingkan per masa dan per komponen antara [39]. Tidak ada voting mayoritas. Setiap kelas ketidaksesuaian ditelusuri ke teks pasal dan diputuskan dalam log adjudikasi tertulis. Log memuat nilai tiap versi, pasal acuan beserta tingkat otoritasnya, putusan, dan versi yang salah. Hasil adjudikasi menjadi *oracle* teradjudikasi untuk RQ2.

Risiko terbesar V2 adalah *common-mode failure*: *knowledge base*, B1, dan adjudikasi ditulis oleh pihak yang sama. Tiga pengaman dipasang terhadap risiko ini: B1 dibekukan sebelum *knowledge base* ditulis; adjudikasi berbasis pasal, bukan berbasis implementasi; dan kalkulator resmi DJP dipakai sebagai penengah eksternal per kelas selisih.

3) V3: metamorphic dan property-based testing: Kebenaran juga diuji lewat relasi yang harus berlaku tanpa memerlukan nilai harapan [9], [38]. Lima belas *metamorphic relation* (MR) dirancang (Tabel IV). Sebagian besar diuji dengan *property-based testing* (pustaka Hypothesis) atas ruang input yang dibangkitkan acak [39], [40]. MR7 dan MR14 diuji dengan contoh tetap, karena masukannya berupa perubahan berkas KB atau tanggal transaksi. MR11 diperiksa sebagai pemeriksaan tipe pada setiap keluaran. MR8 belum diimplementasikan.

4) Kecukupan pengujian dengan mutation testing: Kepekaan *test suite* terhadap kesalahan *knowledge base* diukur dengan *mutation testing* [41], [42]: kesalahan kecil disuntikkan ke KB, lalu dihitung berapa yang tertangkap. Sebanyak 52 mutan dibangkitkan dari operator berikut:

- perubahan tarif dan batas lapisan TER, termasuk *off-by-one*;

Tabel IV
Metamorphic relation (MR) YANG DIRANCANG

MR	Relasi
MR1	Bruto naik $\Rightarrow$ PPh masa tidak turun (*gross*)
MR2	Tanggungan bertambah $\Rightarrow$ PPh setahun tidak naik
MR3	$\sum$ PPh Jan–Des = PPh Pasal 17 setahun (TER)
MR4	*Gross-up*: $T = \mathrm{PPh}_{\mathbb{Z}}(B + T)$ persis
MR5	Memindahkan bonus antarbulan tidak mengubah total setahun
MR6	Memecah satu komponen menjadi dua tidak mengubah hasil
MR7	Menambah aturan di luar tahun t tidak mengubah hasil tahun t
MR8	Memecah berkas kebijakan tidak mengubah hasil (belum diimplementasikan)
MR9	Kategori TER hanya bergantung pada status PTKP
MR10	Neto $\leq$ PTKP $\Rightarrow$ PPh setahun $= 0$
MR11	Setiap fakta uang bertipe bilangan bulat
MR12	Rekonsiliasi buku besar: bruto, neto, dan $\sum$ PPh konsisten
MR13	Mengganti mode pembulatan suatu titik hanya mengubah fakta yang bergantung padanya
MR14	Memindahkan tanggal bayar dalam satu bulan tidak mengubah hasil; melewati batas bulan hanya mengubah masa terkait
MR15	Urutan komponen input tidak memengaruhi hasil

- celah lapisan dan tarif yang tidak monoton;
- perubahan tarif, batas, dan masa berlaku Pasal 17;
- perubahan nilai dan kategori PTKP, daftar KLU, serta parameter;
- perubahan mode dan satuan pembulatan;
- perubahan ekspresi aturan, operator perbandingan, kondisi, dan masa berlaku aturan;
- perubahan klasifikasi wajib dan batas *fixed point*.

Sebuah mutan dinyatakan terbunuh bila tertangkap oleh salah satu detektor: *double-entry* tabel, verifikasi statis, V1, properti V3, atau studi kasus Perusahaan X. Mutan yang lolos semua detektor dibandingkan dengan *golden master*, yaitu keluaran sistem asli pada sampel kasus. Bila keluarannya tidak berubah sama sekali, mutan itu digolongkan *ekuivalen menurut sampel* dan dikeluarkan dari penyebut skor. Bila keluarannya berubah, mutan itu menandakan celah pada *test suite*.

J. Desain Eksperimen dan Metrik

Tabel V memetakan eksperimen ke data, sistem, metrik, dan RQ.

1) Metrik kebenaran: Satuan pemeriksaan adalah tupel (kasus, masa, fakta). Metrik utamanya adalah jumlah *selisih tak terjelaskan* (STJ), yaitu tupel dengan $\hat{y} \neq y$ yang belum memiliki kelas akar masalah, dengan target 0. Kelas akar masalah yang dipakai adalah *bug* KB, erratum contoh resmi, tafsir, *bug* pembanding, dan input tidak valid. Selisih maksimum (MaxAE) dilaporkan per kelas. Tidak ada toleransi: selisih Rp1 diperlakukan sama dengan selisih Rp1 juta sampai kelasnya diketahui. Rata-rata galat (MAE) tidak dipakai sebagai metrik utama karena dapat menyembunyikan kasus yang salah.

2) Metrik adaptabilitas: Untuk setiap skenario perubahan, himpunan S^* (pasangan kasus dan masa yang *seharusnya* berubah) diturunkan secara analitis dari aturan, *sebelum* hasilnya dilihat. Himpunan S adalah pasangan yang benar-benar berubah. Metriknya:

$$P = \frac{|S \cap S^*|}{|S|}, \quad R = \frac{|S \cap S^*|}{|S^*|}, \quad F_1 = \frac{2PR}{P+R}. \qquad (2)$$

Presisi < 1 menandakan efek samping, sedangkan *recall* < 1 menandakan pembaruan yang tidak lengkap. Ukuran pelengkapnya adalah jumlah berkas *engine* yang berubah (target 0), jumlah baris KB yang berubah, dan regresi pada tahun di luar skenario.

3) Metrik deteksi dan ekspresivitas: F1 hanya dipakai untuk tugas yang memang berupa klasifikasi biner, yaitu deteksi konflik antarlapisan dan prediksi keluaran yang terdampak perubahan. Untuk kecukupan uji dipakai *mutation score*: persentase mutan non-ekuivalen yang terbunuh. Ekspresivitas diukur sebagai persentase kebijakan katalog yang dapat dinyatakan pada tiga tingkat: *native* (cukup berkas KB, dieksekusi, dan diperiksa angkanya), cukup lewat isian data, atau tidak bisa tanpa mengubah *engine*.

4) Ablasi: Studi ablasi mematikan satu fitur pada satu waktu:

- A1: versi waktu;
- A2: lapisan perusahaan;
- A3: resolusi konflik;
- A4: penyelesai *fixed point*, diganti iterasi naif;
- A5: registri pembulatan;
- A9: model tiga tanggal.

Untuk setiap varian dilaporkan jumlah tupel yang salah terhadap acuannya, beserta satu contoh tandingan konkret. Dampak aritmetika *float* (A8) ditunjukkan terpisah lewat uji kontrak presisi. Penomoran mengikuti rencana penelitian. Dua ablasi pada rencana itu, yaitu A6 (verifikasi statis) dan A7 (kategori objek wajib), belum diimplementasikan.

5) Penilaian antarmuka: Aplikasi web dan demo dinilai secara fungsional, bukan sebagai bagian dari RQ1–RQ5. Ada tiga hal yang diuji:

- Kasus yang disusun dari basis data identik dengan kasus di dataset.
- Hasil lewat aplikasi identik dengan pemanggilan *engine* langsung.
- Untuk *knowledge base* dasar, cek silang dengan kalkulator tanpa KB identik.

Pengujian ini dijalankan sebagai uji integrasi otomatis di *test suite* aplikasi.

K. Ancaman terhadap Validitas dan Etika

Validitas internal. Penulis *knowledge base* dan B1 adalah pihak yang sama. Selain itu, kode *engine*, berkas KB, B1, transkripsi contoh resmi, dan ekstraksi kedua tabel disusun dengan bantuan asisten pemrograman berbasis LLM di bawah arahan penulis, dan prosesnya tercatat dalam jurnal pengembangan. Asisten yang sama dapat membawa salah paham yang sama ke KB maupun B1, sehingga risiko *common-mode failure* bertambah. Mitigasinya adalah pembekuan B1, adjudikasi berbasis pasal, penengah kalkulator DJP, dan pembaca kedua untuk daftar asumsi. Perlu dicatat bahwa LLM hanya dipakai saat pengembangan. LLM tidak dipakai saat *engine*, *oracle*, atau *baseline* dijalankan.

Hasil uji coba juga sudah memengaruhi KB. Misalnya, satu mutan yang lolos semua detektor mendorong penambahan satu varian tafsir. Karena itu, evaluasi final akan dijalankan

Tabel V
DESAIN EKSPERIMEN

ID	Eksperimen	Data	Metrik utama	RQ
E1	Kebenaran vs contoh resmi	45 kasus kanonik, 1.213 nilai harapan	STJ $= 0$ per tupel; selisih per kelas	RQ2
E2	*Differential testing* KB vs B1	kasus sintetis S1–S7, tanpa *gross-up* rezim 2023 (S8 di E11)	STJ vs *oracle* teradjudikasi; MaxAE per kelas	RQ2
E3	Skenario perubahan regulasi	batas upah JP 2026; DTP berlaku surut (PMK 10/2025); PTKP dan TER hipotetis 2027	berkas *engine* berubah; baris KB berubah; *impact* P/R/F1; regresi	RQ3
E4	Deteksi kesalahan dan konflik	52 mutan KB; pemetaan kebijakan berlabel	*mutation score*; P/R/F1 deteksi konflik	RQ4
E5	Ablasi	acuan per varian (V1, E8, S3, S8)	jumlah tupel salah + contoh tandingan	semua
E6	Analisis *gross-up*	tabel TER A/B/C; strata S3	jumlah dan lebar pita solusi ganda (domain real dan bulat)	RQ5
E7	Ekspresivitas	katalog 20 kebijakan	*coverage* (*native*/*input*/tidak bisa)	RQ1
E8	Studi kasus Perusahaan X	data HR 2023; dihitung ulang 2024–2026	kecocokan per sel vs B0; adjudikasi setiap selisih	RQ2, RQ3
E9	Kinerja	pegawai sintetis	latensi per pegawai-tahun	–
E10	Kalibrasi pembulatan dan tafsir	S7 + kasus perwakilan tiap tafsir	varian yang cocok dengan kalkulator DJP; lebar rentang tafsir	RQ2
E11	Sumbu waktu	strata S8	STJ $= 0$; MR14	RQ2
E12	Kalkulator dengan KB vs tanpa KB	Karyawan A 2023–2026; 8 pegawai sintetis; variasi isian; 45 contoh resmi	identik per rupiah; selisih sah hanya bila terjelaskan	RQ2

pada versi KB yang dibekukan dengan *tag* git, dan setiap perubahan KB sesudah *tag* itu dicatat sebagai temuan. Beberapa keputusan implementasi juga menyimpang dari rancangan awal, dan penyimpangan ini dicatat terbuka: ukuran spesifisitas yang bersifat sintaksis (jumlah konjungsi), ambiguitas yang baru terdeteksi saat evaluasi, verifikasi formal yang belum diimplementasikan, serta penyaringan *knowledge time* yang hanya berlaku bagi aturan bertanggal pencatatan.

Validitas konstruk. Kecocokan total tahunan bukan bukti kebenaran bulanan, sehingga pemeriksaan dilakukan per masa dan per komponen antara. "Jumlah baris yang berubah" hanyalah proksi biaya pemeliharaan, sehingga dilengkapi dengan *impact* F1 dan regresi.

Validitas eksternal. Hanya ada satu studi kasus perusahaan. Mitigasinya adalah katalog 20 kebijakan, kasus sintetis berstrata yang menyasar titik rawan, dan delapan pegawai sintetis yang gajinya diambil dari data UMP dan BPS. Strata S1 belum dikalibrasi ke distribusi upah riil, sehingga persentase dari data sintetis tidak dapat digeneralisasi ke populasi pegawai. Default tafsir belum dikalibrasi ke kalkulator resmi DJP. Sampai kalibrasi E10 selesai, rentang tafsir dapat dihitung lewat CLI.

Etika dan data. Data pribadi pada *spreadsheet* kantor tidak dibaca dan tidak dipublikasikan; hanya turunan teranonimkan yang dirilis. Kalkulator daring hanya diuji secara manual sesuai ketentuan layanannya. Sistem ini adalah prototipe penelitian, bukan alat konsultasi pajak resmi.

DAFTAR PUSTAKA

[1] Direktur Jenderal Pajak, "Peraturan Direktur Jenderal Pajak Nomor PER-16/PJ/2016 tentang Pedoman Teknis Tata Cara Pemotongan, Penyetoran dan Pelaporan Pajak Penghasilan Pasal 21 dan/atau Pajak Penghasilan Pasal 26 Sehubungan dengan Pekerjaan, Jasa, dan Kegiatan Orang Pribadi," 2016, salinan pindaian; diakses 1 Oktober 2026.

[2] Republik Indonesia, "Peraturan Pemerintah Nomor 58 Tahun 2023 tentang Tarif Pemotongan Pajak Penghasilan Pasal 21 atas Penghasilan Sehubungan dengan Pekerjaan, Jasa, atau Kegiatan Wajib Pajak Orang Pribadi," https://peraturan.bpk.go.id/Download/332609/PP%20Nomor%2058%20Tahun%202023.pdf, 2023, diakses 1 Oktober 2026.

[3] Menteri Keuangan Republik Indonesia, "Peraturan Menteri Keuangan Nomor 168 Tahun 2023 tentang Petunjuk Pelaksanaan Pemotongan Pajak atas Penghasilan Sehubungan dengan Pekerjaan, Jasa, atau Kegiatan Orang Pribadi," https://peraturan.bpk.go.id/Download/346093/2023pmkeuangan168.pdf, 2023, diakses 1 Oktober 2026.

[4] ——, "Peraturan Menteri Keuangan Nomor 10 Tahun 2025 tentang Pajak Penghasilan Pasal 21 atas Penghasilan Tertentu yang Ditanggung Pemerintah dalam rangka Stimulus Ekonomi Tahun Anggaran 2025," https://peraturan.bpk.go.id/Download/374407/2025pmkeuangan10.pdf, 2025, diakses 1 Oktober 2026.

[5] ——, "Peraturan Menteri Keuangan Nomor 72 Tahun 2025 tentang Perubahan atas Peraturan Menteri Keuangan Nomor 10 Tahun 2025," https://peraturan.bpk.go.id/Download/394197/72%20TH%202025.pdf, 2025, diakses 1 Oktober 2026.

[6] ——, "Peraturan Menteri Keuangan Nomor 105 Tahun 2025 tentang Pajak Penghasilan Pasal 21 atas Penghasilan Tertentu yang Ditanggung Pemerintah Tahun Anggaran 2026," https://peraturan.bpk.go.id/Download/404917/2025pmkeuangan105.pdf, 2025, diakses 1 Oktober 2026.

[7] D. Merigoux, N. Chataing, and J. Protzenko, "Catala: a programming language for the law," *Proceedings of the ACM on Programming Languages*, vol. 5, no. ICFP, pp. 1–29, 2021, doi: 10.1145/3473582.

[8] D. Merigoux, R. Monat, and J. Protzenko, "A modern compiler for the French tax code," in *Proceedings of the 30th ACM SIGPLAN International Conference on Compiler Construction*. ACM, 2021, pp. 71–82, doi: 10.1145/3446804.3446850.

[9] S. Tizpaz-Niari, V. Monjezi, M. Wagner, S. Darian, K. Reed, and A. Trivedi, "Metamorphic Testing and Debugging of Tax Preparation Software," in *2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering in Society (ICSE-SEIS)*. IEEE, 2023, pp. 138–149, doi: 10.1109/icse-seis58686.2023.00019.

[10] S. Gogani Khiabani, A. Trivedi, D. Saha, and S. Tizpaz-Niari, "An LLM Agentic Approach for Legal-Critical Software: A Case Study for Tax Prep Software," in *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering*. ACM, 2026, pp. 272–283, doi: 10.1145/3744916.3764575.

[11] A. Mowbray, P. Chung, and G. Greenleaf, "Representing legislative Rules as Code: Reducing the problems of 'scaling up'," *Computer Law & Security Review*, vol. 48, p. 105772, 2023, doi: 10.1016/j.clsr.2022.105772.

[12] R. Kowalski and A. Datoo, "Logical English meets legal English for swaps and derivatives," *Artificial Intelligence and Law*, vol. 30, no. 2, pp. 163–197, 2022, doi: 10.1007/s10506-021-09295-3.

[13] S. Dumancic, W. Meert, S. Goethals, T. Stuyckens, J. Huygen, and K. Denies, "Automated Reasoning and Learning for Automated Payroll Management," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 17, pp. 15 107–15 116, 2021, doi: 10.1609/aaai.v35i17.17774.

[14] M. Makowski and J. Trabka, "Dynamic generation of Decision Model and Notation rules for tax regulations – case study from Swiss accounting offices," in *International Conference on Information Systems Development*. University of Gdansk, Department of Business Informatics & University of Belgrade, Faculty of Organizational Sciences, 2025, doi: 10.62036/isd.2025.98.

[15] E. N. Ramadhani, W. Wiranto, and M. K. Giffary, "Perancangan Sistem Perhitungan Pajak Penghasilan Berbasis Web dengan Metode Gross Up," *Riset dan E-Jurnal Manajemen Informatika Komputer (REMIK)*, vol. 6, no. 1, pp. 16–23, 2021, doi: 10.33395/remik.v6i1.11180.

[16] N. L. T. Yulistyawati, N. M. Estiyanti, and S. Lavianto, "Analisis dan Perancangan Sistem Informasi Penggajian Karyawan serta Perhitungan PPh Pasal 21 pada CV Seni Echo," *Jurnal Teknologi Informasi dan Komputer*, vol. 8, no. 4, pp. 306–317, 2022, doi: 10.36002/jutik.v8i4.2084.

[17] M. J. A. B. Kusuma, D. Armeliza, and T. E. Jaya, "Perancangan Sistem Terintegrasi untuk Otomatisasi Perhitungan PPh Pasal 21," *Jurnal Akuntansi, Perpajakan dan Auditing*, vol. 7, no. 1, pp. 85–96, 2026, doi: 10.21009/japa.0701.07.

[18] A. Blair-Stanek, N. Holzenberger, and B. Van Durme, "Can GPT-3 Perform Statutory Reasoning?" in *Proceedings of the Nineteenth International Conference on Artificial Intelligence and Law*. ACM, 2023, pp. 22–31, doi: 10.1145/3594536.3595163.

[19] J. J. Nay, D. Karamardian, S. B. Lawsky, W. Tao, M. Bhat, R. Jain, A. T. Lee, J. H. Choi, and J. Kasai, "Large language models as tax attorneys: a case study in legal capabilities emergence," *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, vol. 382, no. 2270, p. 20230159, 2024, doi: 10.1098/rsta.2023.0159.

[20] M. Dahl, V. Magesh, M. Suzgun, and D. E. Ho, "Large Legal Fictions: Profiling Legal Hallucinations in Large Language Models," *Journal of Legal Analysis*, vol. 16, no. 1, pp. 64–93, 2024, doi: 10.1093/jla/laae003.

[21] L. Huttner and D. Merigoux, "Catala: Moving towards the future of legal expert systems," *Artificial Intelligence and Law*, 2022, doi: 10.1007/s10506-022-09328-5.

[22] R. Monat, A. Fromherz, and D. Merigoux, "Formalizing Date Arithmetic and Statically Detecting Ambiguities for the Law," in *Programming Languages and Systems (ESOP 2024)*, ser. Lecture Notes in Computer Science. Springer Nature Switzerland, 2024, pp. 421–450, doi: 10.1007/978-3-031-57267-8_16.

[23] P. Goutagny, A. Fromherz, and R. Monat, "CUTECat: Concolic Execution for Computational Law," in *Programming Languages and Systems (ESOP 2025)*, ser. Lecture Notes in Computer Science. Springer Nature Switzerland, 2025, pp. 31–61, doi: 10.1007/978-3-031-91121-7_2.

[24] A. Mowbray, P. Chung, and G. Greenleaf, "Explainable AI (XAI) in Rules as Code (RaC): The DataLex approach," *Computer Law & Security Review*, vol. 48, p. 105771, 2023, doi: 10.1016/j.clsr.2022.105771.

[25] P. N. Meessen, "On Normative Arrows and Comparing Tax Automation Systems," in *Proceedings of the Nineteenth International Conference on Artificial Intelligence and Law*. ACM, 2023, pp. 432–436, doi: 10.1145/3594536.3595160.

[26] A. Witt, A. Huggins, G. Governatori, and J. Buckley, "Encoding legislation: a methodology for enhancing technical validation, legal alignment and interdisciplinarity," *Artificial Intelligence and Law*, vol. 32, no. 2, pp. 293–324, 2024, doi: 10.1007/s10506-023-09350-1.

[27] F. Al-Hawari, A. Alufeishat, M. Habahbeh, and A. Alfalayleh, "Scrum-guided development of an intelligent payroll management system with rule-based automation and RAG chatbot support," *Knowledge-Based Systems*, vol. 347, p. 116301, 2026, doi: 10.1016/j.knosys.2026.116301.

[28] A. M. Sherra and N. Fitriani, "Evaluasi Risiko Lebih Bayar PPh 21 Akibat Implementasi Tarif Efektif Rata-Rata (TER) Di CV. X," *Jurnal Wahana Akuntansi*, vol. 20, no. 1, pp. 25–35, 2025, doi: 10.21009/wahana.20.013.

[29] L. Robaldo, S. Batsakis, R. Calegari, F. Calimeri, M. Fujita, G. Governatori, M. C. Morelli, F. Pacenza, G. Pisano, K. Satoh, I. Tachmazidis, and J. Zangari, "Compliance checking on first-order knowledge with conflicting and compensatory norms: a comparison among currently available technologies," *Artificial Intelligence and Law*, vol. 32, no. 2, pp. 505–555, 2024, doi: 10.1007/s10506-023-09360-z.

[30] K. Kharraz, G. Schneider, and M. Leucker, "On Conflicts and Satisfiability in Metric Timed Normative Logics," in *Legal Knowledge and Information Systems (JURIX 2024)*, ser. Frontiers in Artificial Intelligence and Applications. IOS Press, 2024, doi: 10.3233/faia241260.

[31] A. Khoja, M. Kölbl, S. Leue, and R. Wilhelmi, "Automated consistency analysis for legal contracts," *Artificial Intelligence and Law*, vol. 34, no. 3, pp. 881–922, 2026, doi: 10.1007/s10506-025-09456-8.

[32] M. O. Moguillansky, D. C. Martinez, L. H. Tamargo, and A. Rotolo, "An approach to temporalised legal revision through addition of literals," *Artificial Intelligence and Law*, vol. 32, no. 3, pp. 621–666, 2024, doi: 10.1007/s10506-023-09363-w.

[33] H. de Martim, "An Ontology-Driven Graph RAG for Legal Norms: A Hierarchical, Temporal, and Deterministic Approach," in *Legal Knowledge and Information Systems (JURIX 2025)*, ser. Frontiers in Artificial Intelligence and Applications. IOS Press, 2025, doi: 10.3233/faia251598.

[34] F. Olivieri, M. Cristani, G. Governatori, L. Pasetto, A. Rotolo, S. Scannapieco, C. Tomazzoli, and T. Chekole Workneh, "Revising non-monotonic theories with sufficient and necessary conditions: the case of Defeasible Logic," *Journal of Logic and Computation*, vol. 35, no. 7, p. exae044, 2025, doi: 10.1093/logcom/exae044.

[35] A. Boujelben and I. Amous, "Treating Anomalies in Rule bases associated to Ontologies," *The Journal of Supercomputing*, vol. 81, no. 1, p. 243, 2025, doi: 10.1007/s11227-024-06688-4.

[36] W. Fungwacharakorn, K. Tsushima, and K. Satoh, "Resolving counterintuitive consequences in law using legal debugging," *Artificial Intelligence and Law*, vol. 29, no. 4, pp. 541–557, 2021, doi: 10.1007/s10506-021-09283-7.

[37] D. Srinivas, R. Das, S. Tizpaz-Niari, A. Trivedi, and M. L. Pacheco, "On the Potential and Limitations of Few-Shot In-Context Learning to Generate Metamorphic Specifications for Tax Preparation Software," in *Proceedings of the Natural Legal Language Processing Workshop 2023*. Association for Computational Linguistics, 2023, pp. 230–243, doi: 10.18653/v1/2023.nllp-1.23.

[38] R. Li, H. Liu, P.-L. Poon, D. Towey, C.-A. Sun, Z. Zheng, Z. Q. Zhou, and T. Y. Chen, "Metamorphic Relation Generation: State of the Art and Research Directions," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 5, pp. 1–25, 2025, doi: 10.1145/3708521.

[39] H. Goldstein, J. W. Cutler, D. Dickstein, B. C. Pierce, and A. Head, "Property-Based Testing in Practice," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. ACM, 2024, pp. 1–13, doi: 10.1145/3597503.3639581.

[40] S. Ravi and M. Coblenz, "An Empirical Evaluation of Property-Based Testing in Python," *Proceedings of the ACM on Programming Languages*, vol. 9, no. OOPSLA2, pp. 3897–3923, 2025, doi: 10.1145/3764068.

[41] G. Petrovic, M. Ivankovic, G. Fraser, and R. Just, "Practical Mutation Testing at Scale: A view from Google," *IEEE Transactions on Software Engineering*, vol. 48, no. 10, pp. 3900–3912, 2022, doi: 10.1109/tse.2021.3107634.

[42] G. Petrović, M. Ivanković, G. Fraser, and R. Just, "Does Mutation Testing Improve Testing Practices?" in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 2021, pp. 910–921, doi: 10.1109/icse43902.2021.00087.

[43] V. Magesh, F. Surani, M. Dahl, M. Suzgun, C. D. Manning, and D. E. Ho, "Hallucination-Free? Assessing the Reliability of Leading AI Legal Research Tools," *Journal of Empirical Legal Studies*, vol. 22, no. 2, pp. 216–242, 2025, doi: 10.1111/jels.12413.

[44] S. Janatian, H. Westermann, J. Tan, J. Savelka, and K. Benyekhlef, "From Text to Structure: Using Large Language Models to Support the Development of Legal Expert Systems," in *Legal Knowledge and Information Systems (JURIX 2023)*, ser. Frontiers in Artificial Intelligence and Applications, vol. 379. IOS Press, 2023, doi: 10.3233/faia230962.

[45] S. Zuurmond, A. Borg, M. van Kempen, and R. Wieten, "Human-Centred Explanation of Rule-Based Decision-Making Systems in the Legal Domain: Demonstration," in *Legal Knowledge and Information Systems (JURIX 2023)*, ser. Frontiers in Artificial Intelligence and Applications, vol. 379. IOS Press, 2023, doi: 10.3233/faia230992.

[46] Ł. Górski, B. Kuźniacki, M. Almada, K. Tyliński, M. Calvo, P. M. Asnaghi, L. Almada, H. Iñiguez, F. Rubianes, O. Pera, and J. I. Nigrelli, "Exploring explainable AI in the tax domain," *Artificial Intelligence and Law*, vol. 33, no. 3, pp. 551–579, 2025, doi: 10.1007/s10506-024-09395-w.

[47] L. Enqvist, "'Human oversight' in the EU artificial intelligence act: what, when and by whom?" *Law, Innovation and Technology*, vol. 15, no. 2, pp. 508–535, 2023, doi: 10.1080/17579961.2023.2245683.

[48] B. Green, "The flaws of policies requiring human oversight of government algorithms," *Computer Law & Security Review*, vol. 45, p. 105681, 2022, doi: 10.1016/j.clsr.2022.105681.

[49] Republik Indonesia, "Undang-Undang Nomor 7 Tahun 2021 tentang Harmonisasi Peraturan Perpajakan," https://peraturan.bpk.go.id/Download/178620/UU%20Nomor%207%20Tahun%202021.pdf, 2021, diakses 1 Oktober 2026.

[50] Menteri Keuangan Republik Indonesia, "Peraturan Menteri Keuangan Nomor 101/PMK.010/2016 tentang Penyesuaian Besarnya Penghasilan Tidak Kena Pajak," https://peraturan.bpk.go.id/Download/111866/PMK%20Nomor%20101%20Tahun%202016.pdf, 2016, diakses 1 Oktober 2026.